SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCE

Student Information	Details
NAME	G.anushka
REG. NO	1912260668
COURSE	CSA0825 - Python Programming
SLOT	A
GITHUB	https://github.com/anushkapearl21-jpg/CSA0825-co2.git

TASK 1 — Decisions with Many Boundaries

Aim

To analyze electricity subsidy rules under multi-boundary consumption conditions and evaluate separate `if` versus `if-elif-else` execution paths.

Concept / Theory

In Python, using separate independent `if` statements causes every condition to be evaluated sequentially. If an input satisfies multiple boundaries, multiple branches execute, resulting in bugged duplicate outputs. Conversely, an if-elif-else structure guarantees mutually exclusive execution paths where exactly one decision is returned.

Procedure / Algorithm

1. Identify state electricity consumption slab boundaries (`<= 100`, `101-200` with ration card, `> 200` with senior citizen status).
2. Predict decisions for 8 representative households across boundary conditions.
3. Implement the decision logic using separate `if` statements.
4. Implement the decision logic using mutually exclusive `if-elif-else` statements.
5. Compare execution behavior and analyze time/space complexity using Big-O notation.

Program

Python

```python
def evaluate_subsidy_separate_if(units, has_ration_card, has_senior):
    results = []
    if units <= 100:
        results.append("Full Subsidy")
    if 101 <= units <= 200 and has_ration_card:
        results.append("Partial Subsidy")
    if units > 200 and has_senior:
        results.append("Special Subsidy (Senior)")
    if units > 200 and not has_senior:
        results.append("No Subsidy")
    return results

def evaluate_subsidy_if_elif(units, has_ration_card, has_senior):
    if units <= 100:
        return "Full Subsidy"
    elif 101 <= units <= 200:
        if has_ration_card:
            return "Partial Subsidy"
        else:
            return "No Subsidy"
    else:
        if has_senior:
            return "Special Subsidy (Senior)"
        else:
            return "No Subsidy"
```

Analysis / Discussion

Separate `if` statements double-count decisions whenever multiple conditions evaluate to True. The `if-elif-else` structure provides mutually exclusive branching. Time Complexity is O(1) per evaluation as conditions execute in constant steps. Space Complexity is O(1) auxiliary memory.

Result

Thus, the multi-boundary electricity subsidy decision logic was implemented and evaluated, proving that if-elif-else ensures single mutually exclusive decisions.

TASK 2 — Loops that Count Attempts

Aim

To design an OTP attempt counter loop at a bank kiosk using while and for loops, handling blank inputs and lockouts.

Concept / Theory

A while loop is naturally suited for event-driven input verification where the total number of iterations cannot be predetermined due to blank or invalid retries. Placing attempt increments after blank-input checks prevents invalid entries from unfairly exhausting attempt limits.

Procedure / Algorithm

1. Define the correct target OTP (`'1234'`) and maximum permitted attempts (3).
2. Construct a `while` loop that reads user input and ignores blank entries without incrementing the counter.
3. Implement an alternative loop using `for _ in range()` with conditional `break` triggers.
4. Incorporate a loop-`else` clause to lock the account for 30 minutes upon 3 failed attempts.
5. Analyze recursive vs iterative implementations regarding call stack space usage.

Program

Python

```python
def verify_otp_while():
    correct_otp = "1234"
    attempts = 0
    max_attempts = 3

    while attempts < max_attempts:
        user_input = input("Enter OTP: ").strip()
        if user_input == "":
            print("Blank entry detected. Not counted as attempt.")
            continue

        if user_input == correct_otp:
            print("OTP Verified Successfully!")
            break
        else:
            attempts += 1
            print(f"Incorrect OTP. Attempts remaining: {max_attempts - attempts}")
    else:
        print("Account LOCKED for 30 minutes due to 3 failed attempts.")

def verify_otp_for():
    correct_otp = "1234"
    attempts_counted = 0

    for _ in range(100):
        if attempts_counted >= 3:
            print("Account LOCKED for 30 minutes.")
            break
        user_input = input("Enter OTP: ").strip()
        if user_input == "":
            continue
        if user_input == correct_otp:
            print("OTP Verified Successfully!")
            break
        attempts_counted += 1
```

Analysis / Discussion

The `while` loop is more natural than a fixed range `for` loop because user retries (blank inputs) do not advance attempt state. Updating attempt counters before checking blank inputs creates a bug. Recursion requires O(a) call stack space for a attempts, whereas iterative loops use O(1) auxiliary space.

Result

Thus, the bank kiosk OTP verification loop was implemented, correctly validating attempts, ignoring blank entries, and enforcing account lockout.

TASK 3 — Engineer a Vehicle Registration Number Validator

Aim

To build a toll plaza vehicle registration number plate normalizer and validator using string slicing and methods without regular expressions.

Concept / Theory

Strings in Python are immutable sequence objects. String transformation methods return newly created string objects rather than modifying existing instances in memory. Slicing with negative indices allows extracting fixed-length terminal sequences such as standard 4-digit registration numbers.

Procedure / Algorithm

1. Strip whitespace and dashes from raw input strings and convert characters to uppercase.
2. Validate total character length (must equal 9 or 10 characters).
3. Extract components using string slicing: state code (indices 0:2), district code (2:4), number (-4:), and series (4:-4).
4. Verify character types using `isalpha()` and `isdigit()` methods.
5. Format valid strings into receipt standard format `'TN 09 AB 1234'`.

Program

```python
def validate_and_format_plate(raw_plate):
    clean = raw_plate.replace("-", "").replace(" ", "").upper()

    if len(clean) not in [9, 10]:
        return False, "Invalid Length"

    state = clean[:2]
    district = clean[2:4]
    number = clean[-4:]
    series = clean[4:-4]

    if state.isalpha() and district.isdigit() and series.isalpha() and
number.isdigit():
        formatted = f"{state} {district} {series} {number}"
        return True, formatted
    return False, "Invalid Format"
```

Analysis / Discussion

Strings are immutable, so `text.upper()` returns a new string rather than changing `text` in place. For a 9-char string `'TN09A1234'`, `s[-4:]` returns `'1234'` and `s[4:-4]` returns 'A'. For a 10-char string 'TN09AB1234', s[-4:] returns '1234' and s[4:-4] returns 'AB'.

Result

Thus, the toll plaza vehicle registration plate validator was engineered and verified across multiple input formats using Python string methods.

TASK 4 — Performance Analysis of String Concatenation

Aim

To compare the execution time and complexity of string building using repeated `+=` operators versus `str.join()`.

Concept / Theory

Because Python strings are immutable, appending to a string using += allocates a new memory block and copies all previously concatenated characters on each step. This results in $O(n^2)$ quadratic time complexity. Conversely, str.join() calculates total required memory up front, yielding O(n) linear time.

Procedure / Algorithm

1. Generate a list of 200,000 string elements.
2. Build concatenated string using a `for` loop with repeated `+=` operators.
3. Build identical string using a single `str.join()` call.
4. Measure precise execution time using `time.perf_counter()`.
5. Verify string equality and output timing results.

Program

Python

```python
import time

items = [f"Item{i}:Rs{i % 50}" for i in range(200_000)]

def build_plus(parts):
    text = ""
    for p in parts:
        text += p + ";"
    return text

def build_join(parts):
    return ";".join(parts) + ";"

start = time.perf_counter()
a = build_plus(items)
t_plus = time.perf_counter() - start

start = time.perf_counter()
b = build_join(items)
t_join = time.perf_counter() - start
```

```
print("same text:", a == b, "| length:", len(b))
print(f"+= time: {t_plus:.4f} s | join time: {t_join:.4f} s")
```

Analysis / Discussion

The repeated += loop exhibits O(n²) quadratic complexity due to continuous reallocations and copying. In contrast, `str.join()` operates in O(n) linear time and linear space, drastically outperforming loop concatenation on large datasets.

Result

Thus, performance analysis proved str.join() significantly faster than += string concatenation for accumulating large sequence data.

TASK 5 — Functions, Parameters and Scope

Aim

To analyze mutable default parameter bugs in Python functions and structure clean order processing logic.

Concept / Theory

Default parameter values in Python functions are evaluated once when the function definition is executed, not each time the function is called. Using a mutable object like a list (`[]`) as a default value causes modifications to persist across calls. Initializing default arguments to None solves this bug.

Procedure / Algorithm

1. Demonstrate bugged behavior of default mutable argument `order=[]`.
2. Correct the function by initializing `order=None` and creating a fresh list inside the call scope.
3. Test functions with sequential customer calls to ensure order independence.

4. Explain parameter types (positional, keyword, default) and variable scopes (local, global, nonlocal).

Program

```python
def add_item_buggy(item, order=[]):
    order.append(item)
    return order

def add_item(item, order=None):
    if order is None:
        order = []
    order.append(item)
    return order

print("Buggy Output :", add_item_buggy("Tea"), add_item_buggy("Vada"))
print("Corrected Output:", add_item("Tea"), add_item("Vada"))
```

Analysis / Discussion

Mutable default arguments share state across function calls because the default list is bound at definition time. Returning multiple items is achieved using tuple packing. Modifying global variables inside functions requires the `global` keyword but is avoided to prevent hidden state coupling.

Result

Thus, the canteen order function was corrected by eliminating mutable default arguments, ensuring independent customer order states.

TASK 6 — Recursion and Its Limits

Aim

To evaluate naive recursive versus memoized solutions for temple step counting and analyze call-stack limits.

Concept / Theory

Recursive algorithms decompose problems into smaller self-similar subproblems. Naive recursive Fibonacci implementations recompute subproblems repeatedly, causing exponential $O(2^n)$ call trees. Memoization via `functools.lru_cache` stores computed subproblem outputs, caching results to achieve O(n) execution time.

Procedure / Algorithm

1. Implement a naive recursive function for step combinations ($f(n) = f(n-1) + f(n-2)$).
2. Apply `@lru_cache(maxsize=None)` to create a memoized variant.
3. Count total function calls for both approaches across inputs $n = 10, 20, 25$.
4. Observe call stack exhaustion (`RecursionError`) for large $n = 1000$ under `sys.getrecursionlimit()`.

Program

Python

```python
from functools import lru_cache
import sys

calls = 0

def ways_naive(n):
    global calls
    calls += 1
    if n <= 1:
        return 1
    return ways_naive(n - 1) + ways_naive(n - 2)

@lru_cache(maxsize=None)
def ways_memo(n):
    global calls
    calls += 1
    if n <= 1:
        return 1
    return ways_memo(n - 1) + ways_memo(n - 2)

calls = 0; print("naive:", ways_naive(25), "calls =", calls)
```

```
calls = 0; print("memo :", ways_memo(25), "calls =", calls)
```

Analysis / Discussion

Naive recursion takes $O(2^n)$ time and 242,785 call frames for $n=25$. Memoized recursion reduces call count to 26 calls, taking O(n) time. However, memoization still consumes O(n) stack depth memory, causing RecursionError for $n=1000$ unless rewritten iteratively.

Result

Thus, memoization was shown to eliminate redundant recursive calculations, while iterative transformation was identified as necessary for deep call sequences.

TASK 7 — Diagnose, Redesign and Validate

Aim

To diagnose performance bottlenecks in recursive step counting, redesign the algorithm iteratively, and defend the optimized design.

Concept / Theory

While memoization optimizes time complexity to O(n), it retains linear O(n) call stack space. An iterative dynamic programming approach maintains only the prior two state values in scalar variables, reducing space complexity from O(n) call-stack depth down to O(1) auxiliary memory.

Procedure / Algorithm

1. **Diagnose Bottleneck:** Naive recursion recomputes subproblems exponentially $O(2^n)$; memoization consumes O(n) stack frame space.
2. **Define Design Objective:** Achieve O(n) linear time complexity with O(1) constant auxiliary space, supporting $n=1000$ without recursion errors.
3. **Design Revised Solution:** Construct a bottom-up iterative loop tracking two running scalar variables.

4. **Implement and Validate:** Execute iterative version for $n=1000$ and verify exact integer results in sub-millisecond time.
5. **Defend Design:** Compare execution metrics across versions to confirm optimal space/time efficiency.

Program

```
n
def ways_iterative(n):
if n <= 1:
return 1
prev2, prev1 = 1, 1
for _ in range(2, n + 1):
prev2, prev1 = prev1, prev2 + prev1
return prev1

result_1000 = ways_iterative(1000)
print(f"Iterative n=1000 completed successfully.")
print(f"Result digit count: {len(str(result_1000))} digits")
```

```
None

## Analysis / Discussion
The iterative redesign successfully eliminates all stack-frame overhead. Execution
time scales linearly in O(n) while space complexity remains strictly O(1)
auxiliary space, making it the most robust production implementation.

## Result
Thus, the recursive bottleneck was diagnosed and successfully redesigned into an
iterative O(n) time, O(1) space solution that executes seamlessly for $n=1000$.
```